

Deterministic Adjudication Lattices and Evolutionary Convergence in Constrained Generative Agent Populations

the.PM

Iberian Peninsula Human Civilization Continuation Project

New Lisbon Agency

Correspondence: support@newlisbon.agency

Submitted to: cs.AI (cross-listed cs.FL, cs.SE)

April 2026

Authorship commitment: This document was timestamped via OpenTimestamps. The matching .ots proof is available upon authorship dispute claim.

Abstract

Generative agents produce structured outputs with no formal guarantee of correctness. Prevailing approaches to output validation, including reinforcement learning from human feedback, static guardrail classifiers and single-pass output filtering, are inherently probabilistic and furnish no convergence proof. We introduce the *adjudication lattice* framework, in which an evolutionary population of candidate outputs is iteratively filtered through a hierarchical lattice of deterministic verification predicates. A secondary discriminator evaluates fitness among survivors. We prove that under mild conditions on the lattice structure and population dynamics, this process converges to a zero-defect output in bounded generations, with population size logarithmic in the inverse failure probability.

We further introduce the *centroid absorber*, a continuity-preservation mechanism that maintains coherence across sequential outputs from a single generative process, with provable drift bounds independent of sequence length.

Finally, we formalize a *freshness predicate* over the agent's epistemic state that prevents action on stale context, cast as a liveness condition in a concurrent verification model. Together, these contributions define a unified framework in which the probability of a defective output decreases exponentially with population size and lattice depth, at polynomial computational cost.

Keywords: formal verification, evolutionary computation, constraint satisfaction, generative agents, oracle machines, lattice theory, sequential coherence, temporal logic

1 Introduction

1.1 The Verification Gap in Generative Systems

The past decade has witnessed the rapid proliferation of generative models capable of producing complex structured outputs: natural-language text, executable code, structured data conforming to schemas and multi-modal artifacts combining several of these modalities. The statistical foundations of such models guarantee that outputs are *likely* in the sense of high probability under a learned distribution, yet they provide no formal guarantee that any given output is *correct* with respect to a deterministic specification. This gap between probabilistic generation and deterministic verification constitutes one of the central open problems in trustworthy artificial intelligence.

The formal methods community has developed a rich arsenal of techniques for verifying the correctness of deterministic artifacts: model checking [1], theorem proving [2], abstract interpretation [3] and static analysis [4]. These techniques operate on a fundamentally different contract than generative models. A model checker either confirms that a system satisfies a temporal logic specification or produces a counterexample; there is no probability of error (up to the soundness of the tool). A generative model, by contrast, samples from a distribution and returns whatever the sample produces. The verification gap is not merely quantitative (low error rate versus zero error rate) but qualitative: the contract offered by the two paradigms is of an altogether different character.

The alignment and safety literature in AI has recognized this gap under various names. The hallucination problem in large language models [5, 6] is a symptom: the model generates text that is fluent and plausible but factually incorrect. Output classifiers and guardrail systems [7, 8] attempt to filter problematic outputs post hoc, but as we argue in the next subsection, single-pass filtering cannot provide convergence guarantees. Reinforcement learning from human feedback (RLHF) [9, 10] shifts the distribution toward preferred outputs, yet the resulting distribution still assigns nonzero probability to defective outputs. No existing approach combines deterministic verification with a generative process in a manner that yields provable convergence to zero-defect outputs.

1.2 Why Post-Hoc Filtering is Insufficient

Consider a generative process G that produces outputs, and a filter f that accepts or rejects each output based on some correctness criterion. Suppose f has a false-negative rate (the probability that a defective output passes the filter) of $\varepsilon > 0$. A single application of f to a single output leaves a residual defect probability of ε .

One might attempt to cascade multiple independent filters $f_1, f_2, \dots, f_k$. If their false-negative events are independent, the residual defect probability after k filters is $\varepsilon_1 \cdot \varepsilon_2 \cdots \varepsilon_k$. This product approaches zero as k grows, but it never reaches zero in finite composition. More critically, the

independence assumption is unrealistic: filters checking related properties will have correlated failures. A structural validator and a semantic validator may both fail on the same pathological input. The product bound therefore overstates the actual defect reduction.

Furthermore, filtering alone provides no gradient toward correctness. If every candidate output from G is defective, no amount of filtering will produce a correct output; the filter merely rejects all candidates and returns nothing. A purely filtering-based approach lacks the evolutionary pressure required to drive the generative process toward the feasible region of the output space.

1.3 The Adjudication Lattice Intuition

The core idea of this paper is to replace single-output filtering with population-based evolutionary selection under hierarchical deterministic verification. Instead of generating one output and checking it, we generate a *population* of N candidate outputs and pass all of them through a *adjudication lattice* of increasingly strict verification predicates. The lattice is organized as a lattice rather than a simple linear sequence, because some verification predicates are orthogonal (checking independent properties) rather than hierarchical (one strictly implying the other). Natural selection operates on the population: candidates that fail any predicate in the lattice are eliminated. Among the survivors, a discriminator (fitness function) selects the highest-quality output.

This construction draws on two classical traditions. From formal verification, it inherits the determinism and soundness of verification predicates: if the predicates are sound, no defective output can survive the lattice. From evolutionary computation, it inherits the population-based search strategy: even if any single candidate is unlikely to pass the entire lattice, a sufficiently large population ensures that at least one candidate does. The marriage of these traditions yields something neither can provide alone: provable convergence to a zero-defect output with explicit population-size bounds.

1.4 Contribution Summary

This paper makes three formal contributions:

1. **The Adjudication Lattice framework and convergence theorem.** We define a hierarchical lattice of deterministic verification predicates and prove that constrained evolutionary selection over a population of generative outputs converges to a zero-defect output with probability at least $1 - \delta$ for any target failure probability $\delta > 0$, requiring a population of size $N = O(\ln(1/\delta))$.
2. **The Centroid Absorber for sequential coherence.** We introduce a constant-time coherence validation mechanism for sequential outputs, prove that centroid drift is bounded by $O(H_t)$ (the harmonic series) under centroid-only validation and by $O(1)$ under joint anchor-centroid validation and show that the absorber composes naturally with the adjudication lattice as an additional verification predicate.

3. **The Freshness Predicate for temporal epistemic consistency.** We formalize a liveness condition over the agent's epistemic state that prevents generation from stale context, prove that it provides bounded temporal lag guarantees in concurrent multi-agent settings and show that it is orthogonal to all structural and semantic predicates, adding an independent dimension to the adjudication lattice at no asymptotic cost.

1.5 Paper Organization

Section 2 establishes the mathematical preliminaries and notation. Section 3 develops the adjudication lattice framework and states the main convergence theorem. Section 4 introduces the centroid absorber and proves its drift bounds. Section 5 formalizes the freshness predicate as a liveness condition. Section 6 presents the hierarchical generation strategy with speculative generation and amortized cost analysis. Section 7 describes experimental validation on synthetic adjudication lattices. Section 8 surveys related work. Section 9 discusses limitations and open problems. Section 10 concludes. Appendices A through C contain full proofs, a detailed predicate lattice construction for structured outputs and the category-theoretic perspective.

2 Preliminaries and Notation

2.1 Generative Processes

Definition 2.1 (Generative Process).

A generative process is a stochastic map $G: S \times Z \rightarrow O$, where S is a specification space (the set of all valid prompts, schemas or task descriptions), Z is a latent noise space equipped with a probability measure ν , and O is the output space. Each invocation $G(s, z)$ for $s \in S$, $z \sim \nu$ produces a candidate output $o \in O$.

The specification s encodes the deterministic requirements that an output must satisfy. The latent variable z captures the stochastic element of generation. We write $G(s)$ to denote the random variable obtained by marginalizing over z , so that $G(s)$ has distribution μ_s on O induced by the pushforward of ν through $G(s, \cdot)$.

Definition 2.2 (Defect Function).

The defect function $d: O \times S \rightarrow \{0, 1\}^k$ maps each output-specification pair to a binary vector of length k , where k is the number of defect categories. The i -th component $d_i(o, s) = 1$ indicates the presence of a defect of type i . An output o is zero-defect with respect to specification s if and only if $d(o, s) = 0^k$, the zero vector.

The defect categories are application-dependent. Typical categories include structural invalidity (the output does not conform to the required schema), semantic incorrectness (the output is well-formed but violates domain constraints), coherence failure (the output is inconsistent with prior outputs in a sequence) and temporal staleness (the output was generated from an outdated context). We assume that the defect function is well defined for all output-specification pairs, though it need not be efficiently computable.

2.2 Verification Predicates

Definition 2.3 (Verification Predicate).

A verification predicate is a total, deterministic function $v: O \times S \rightarrow \{\text{accept}, \text{reject}\}$. Totality requires that v terminates on every input. Determinism requires that $v(o, s)$ is uniquely determined by (o, s) .

The totality requirement distinguishes verification predicates from general decision procedures, which may diverge on certain inputs. In practice, totality is ensured by imposing resource bounds (time and space limits) on the predicate's execution, with timeout treated as rejection. The determinism requirement ensures that the predicate's behaviour is reproducible: given the same output and specification, the predicate always returns the same verdict.

Definition 2.4 (Soundness).

A verification predicate v is sound with respect to defect category i if for all $o \in O$ and $s \in S$, $v(o, s) = \text{accept}$ implies $d_i(o, s) = 0$. That is, a sound predicate never accepts a defective output (with respect to the defect category it guards).

Soundness is a one-directional guarantee. A sound predicate may reject correct outputs (false positives), but it will never accept defective outputs (false negatives). This asymmetry is deliberate: we prefer a conservative predicate that occasionally rejects valid candidates over a permissive predicate that sometimes admits defective ones. The false-positive rate affects efficiency (larger populations are needed) but not correctness.

Definition 2.5 (Predicate Lattice).

The predicate lattice is a partially ordered set $(V, \leq)$ where V is a set of verification predicates and $v_1 \leq v_2$ if and only if for all $o \in O$, $s \in S$: $v_2(o, s) = \text{accept}$ implies $v_1(o, s) = \text{accept}$. That is, v_2 is at least as strict as v_1 .

Note. The structure $(V, \leq)$ as defined above is guaranteed to have meets (the conjunction of any two predicates is again a verification predicate, by Proposition 2.1), but joins (disjunctions) may not be meaningful for verification purposes. Strictly speaking, $(V, \leq)$ is therefore a meet-semilattice rather than a full lattice in the algebraic sense. The term "lattice" is retained throughout this paper for continuity with the "adjudication lattice" terminology introduced in Definition 2.6.

Intuitively, higher elements in the lattice impose stricter requirements. If $v_2 \leq v_1$ fails and $v_1 \leq v_2$ fails, then v_1 and v_2 are incomparable: they check orthogonal properties. The lattice structure captures both hierarchical and orthogonal relationships among predicates in a single algebraic framework.

2.3 Adjudication Lattices

Definition 2.6 (Adjudication Lattice).

An adjudication lattice $T = (v_1, v_2, \dots, v_n)$ is a finite sequence of verification predicates drawn from the predicate lattice $(V, \leq)$. For the canonical form, we require that $v_1 \leq v_2 \leq \dots \leq v_n$, so that v_1 is the weakest predicate (e.g., schema validation) and v_n is the strongest (e.g., full formal verification). The generalized form permits arbitrary elements of V , including incomparable (orthogonal) predicates.

Definition 2.7 (Lattice Filter).

The lattice filter $T: O \times S \rightarrow \{\text{accept}, \text{reject}\}$ is defined by $T(o, s) = \text{accept}$ if and only if $v_i(o, s) = \text{accept}$ for all $i \in \{1, \dots, n\}$. That is, the lattice filter accepts an output only if every predicate in the lattice accepts it.

Proposition 2.1 (Closure under Conjunction).

The lattice filter T is itself a verification predicate. Moreover, if every v_i in the lattice is sound with respect to defect category c_i , then T is sound with respect to all categories $\{c_1, \dots, c_n\}$.

Proof.

Totality: since each v_i is total, the conjunction terminates for every input. Determinism: since each v_i is deterministic, the conjunction is deterministic. Soundness: suppose $T(o, s) = \text{accept}$. Then $v_i(o, s) = \text{accept}$ for all i , whence $d_{\{c_i\}}(o, s) = 0$ for all i by soundness of the individual predicates. Thus o is free of all defects in $\{c_1, \dots, c_n\}$.

Y

2.4 Population Dynamics

Definition 2.8 (Candidate Population).

A candidate population at generation t is a multiset $P_t = \{o_1, o_2, \dots, o_N\}$ of N outputs drawn independently from the generative process $G(s)$. We write $P_t \sim G(s)^N$ to denote N independent draws.

Definition 2.9 (Selection Operator).

The selection operator $\text{Sel}: 2^O \times T \times F \rightarrow O \cup \{\perp\}$ takes a candidate population, a lattice filter and a fitness function, and returns the highest-fitness output among those that survive the lattice filter. If no candidate survives, Sel returns $\perp$ (failure). Formally, let $S_t = \{o \in P_t : T(o, s) = \text{accept}\}$. If $S_t = \emptyset$, then $\text{Sel}(P_t, T, F) = \perp$. Otherwise, $\text{Sel}(P_t, T, F) = \text{argmax}_{\{o \in S_t\}} F(o, s)$.

2.5 Measure-Theoretic Setup

Let μ_s denote the probability measure on O induced by the generative process $G(s)$. For each predicate v_i in the lattice, define the acceptance probability:

$$\alpha_i = \Pr_{\{o \sim \mu_s\}}[v_i(o, s) = \text{accept}].$$

The joint acceptance probability of the lattice is:

$$\alpha_T = \Pr_{\{o \sim \mu_s\}}[T(o, s) = \text{accept}] = \Pr_{\{o \sim \mu_s\}}[\bigwedge_i v_i(o, s) = \text{accept}].$$

In general, α_T is not the product of the individual α_i values because the acceptance events may be correlated. We have $\alpha_T \leq \min_i \alpha_i$ always, and $\alpha_T = \prod_i \alpha_i$ only when the acceptance events are mutually independent. The relationship between orthogonality in the predicate lattice and statistical independence is a central concern of Section 3.

2.6 Category-Theoretic Framing

For readers familiar with category theory, we offer an alternative perspective that illuminates the algebraic structure of the adjudication lattice. Define the category **Ver** whose objects are verification predicates and whose morphisms are implication relationships: a morphism $v_1 \rightarrow v_2$ exists if and only if $v_1 \leq v_2$ in the predicate lattice. This category is a thin category (at most one morphism between any two objects) and is equivalent, as a poset, to $(V, \leq)$.

The adjudication lattice $T = (v_1, \dots, v_n)$ defines a finite diagram $D: I \rightarrow \mathbf{Ver}$, where I is the indexing category (a finite discrete category or a finite chain, depending on whether the lattice is ordered). The lattice filter T is the limit (specifically, the product in the thin-category sense) of this diagram. Acceptance by T is equivalent to membership in the limit cone.

The population dynamics define a functor $\Phi: \mathbf{N} \rightarrow \mathbf{FinSet}(O)$ from the category of natural numbers (with the usual order) to the category of finite subsets of O . At each generation t , $\Phi(t) = P_t$ is the candidate population. The selection operator Sel is a natural transformation from Φ to the constant functor at O , mediated by the lattice filter and fitness function. This functorial perspective is developed fully in Appendix C.

3 The Adjudication Lattice Framework

3.1 Lattice Composition and Orthogonality

Not all verification predicates are comparable in the lattice. A predicate checking structural validity (e.g., JSON schema conformance) and a predicate checking semantic correctness (e.g., domain constraint satisfaction) may be entirely independent: an output can pass one while failing the other, in either direction. We formalize this observation.

Definition 3.1 (Orthogonality).

Two verification predicates v_i and v_j are orthogonal, written $v_i \perp v_j$, if neither $v_i \leq v_j$ nor $v_j \leq v_i$. Equivalently, there exist outputs that are accepted by v_i but rejected by v_j , and outputs that are accepted by v_j but rejected by v_i .

Theorem 3.1 (Exponential Defect Reduction under Orthogonality).

Let $T = (v_1, \dots, v_k)$ be an adjudication lattice in which the k predicates are pairwise orthogonal (Definition 3.1). Then, for any $n \geq 1$, the number of outputs accepted by T is at most 2^n .

population cost grows only as $N = O(\ln(1/\delta) / \alpha^k)$, and every survivor is guaranteed zero-defect across all k categories. The lattice thus converts computational cost (larger N) into correctness breadth (more defect categories covered), a trade-off unavailable to monolithic verification.

Observation 3.1 (Correlated predicates). The exponential bound of Theorem 3.1 depends on the positive conditional association assumption, which in turn depends on the orthogonality of the predicates. In practice, logical orthogonality (incomparability in the lattice) does not automatically guarantee statistical independence under a given generative distribution. Syntactic correctness and semantic correctness, for instance, are logically orthogonal (an output can satisfy either without the other) yet may be positively correlated under a well-trained generator: outputs that are syntactically well-formed tend also to be semantically closer to correct. When predicates are positively correlated, the joint acceptance probability α_T exceeds the product $\prod_i \alpha_i$, and the defect reduction is sub-exponential in k . For an equicorrelated multivariate Bernoulli model with pairwise correlation $\rho \geq 0$ between acceptance events, the joint acceptance probability satisfies $\alpha_T \geq \alpha^k$ (the uncorrelated lower bound does not apply; positive correlation increases the joint probability). A simple upper bound on the joint defect probability under equicorrelation is given by the Frechet-Hoeffding bound: $1 - \alpha_T \leq k(1 - \alpha)$ (the union bound, which holds regardless of correlation). The precise joint probability depends on the full correlation structure and no closed-form expression exists for general k and ρ . Practitioners should estimate α_T empirically rather than relying on the product bound when predicates are suspected to be correlated.

3.2 Stratum Structure

Definition 3.2 (Strata).

A stratum in the adjudication lattice is an equivalence class of predicates at the same level in the lattice. Formally, define the equivalence relation $v_i \sim v_j$ if $v_i \leq v_j$ and $v_j \leq v_i$ (i.e., they accept exactly the same outputs). The strata form a quotient poset $(V/\sim, \leq)$ that inherits the graded structure of the lattice.

Definition 3.3 (Stratum Rejection Function).

The stratum rejection function $r_j(P)$ gives the fraction of population P rejected at stratum j but accepted at all strata below j . Formally, $r_j(P) = |\{o \in P : v_j(o, s) = \text{reject} \wedge \bigwedge_{i < j} v_i(o, s) = \text{accept}\}| / |P|$.

Theorem 3.2 (Monotone Stratum Rejection).

If the generative process conditions on prior accepted outputs (that is, at generation $t+1$ the generator has access to the surviving outputs from generation t as exemplars), then the stratum rejection function r_j is monotonically non-increasing across generations for each stratum j . That is, $r_j(P_{t+1}) \leq r_j(P_t)$ in expectation.

Proof.

Conditioning on prior accepted outputs shifts the generative distribution toward the feasible region. Formally, let μ_t denote the distribution at generation t and let μ_{t+1} denote the conditioned distribution. By construction, μ_{t+1} assigns higher probability to outputs that passed stratum j at generation t . Since r_j measures the mass in the region that passes all strata below j but fails at j , and the conditioning increases the mass in the region that passes all strata including j , the rejection fraction at stratum j decreases in expectation. A formal proof by induction on the number of conditioning steps, using the data processing inequality for the KL divergence between μ_t and the uniform distribution on the feasible set, is given in Appendix A.

Y

Observation 3.2 (Scope of the conditioning assumption). Theorem 3.2 assumes that the generative process can be effectively conditioned on prior accepted outputs such that the resulting distribution shifts monotonically toward the feasible region. This assumption holds exactly for ideal Bayesian conditioning and for specific evolutionary algorithms (e.g., CMA-ES, which adapts its covariance matrix from successful samples). For generative processes based on in-context learning (e.g., providing accepted outputs as few-shot exemplars to a language model), the assumption is an approximation. In-context learning does not perform exact Bayesian conditioning; it is mediated by the attention mechanism and may exhibit non-monotonic behaviour due to context window saturation, mode collapse under narrow exemplar distributions or catastrophic interference when exemplars conflict with pretrained priors. We therefore treat Theorem 3.2 as a best-case characterization of conditioned generation. The convergence theorem (Theorem 3.3), which is the primary result, does not depend on Theorem 3.2: it requires only $\alpha_T > 0$ at a single generation, not monotonic improvement across generations. Theorem 3.2 provides an efficiency refinement (decreasing N over time) rather than a correctness guarantee.

3.3 The Convergence Theorem

We now state the main result of the paper.

Theorem 3.3 (Convergence of Constrained Evolutionary Selection).

Let G be a generative process with specification s such that the joint acceptance probability $\alpha_T = \Pr\{o \sim G(s) \mid [T(o, s) = \text{accept}]\} > 0$. Let N be the population size per generation. Then the probability that the selection operator Sel returns a zero-defect output satisfies:

$$\Pr[\text{Sel}(P, T, F) \neq \perp] \geq 1 - (1 - \alpha_T)^N.$$

In particular, for any target failure probability $\delta > 0$, setting $N = \lceil \ln(\delta) / \ln(1 - \alpha_T) \rceil$ suffices to guarantee $\Pr[\text{Sel returns a zero-defect output}] \geq 1 - \delta$.

Proof.

Each of the N candidates in the population is drawn independently from $G(s)$. The probability that a single candidate fails the lattice filter is $1 - \alpha_T$. The probability that all N candidates fail is $(1 - \alpha_T)^N$ by independence. Therefore, the probability that at least one candidate survives is $1 - (1 - \alpha_T)^N$. If at least one candidate survives the lattice filter, and if the lattice is sound, then the surviving candidate is zero-defect by Proposition 2.1. Sel selects the highest-fitness survivor, which is also zero-defect. The population size bound follows from solving $1 - (1 - \alpha_T)^N \geq 1 - \delta$ for N , yielding $N \geq \ln(\delta)/\ln(1 - \alpha_T)$.

Y

Corollary 3.3.1 (Single-Generation Sufficiency).

For any target failure probability $\delta > 0$, a finite population size N suffices to achieve zero-defect selection with probability $1 - \delta$ in a single generation. No iterative refinement is required if N is large enough.

Corollary 3.3.2 (Self-Improving Dynamics).

When the generative process conditions on prior accepted outputs (cf. Theorem 3.2), the joint acceptance probability α_T increases monotonically across generations. Consequently, the population size required to achieve a given failure probability δ decreases over successive generations. The process is self-improving.

3.4 The Fitness Discriminator

The lattice filter guarantees correctness: any output that survives the lattice is zero-defect. The fitness function F selects for quality among correct outputs. We now formalize the discriminator and its interaction with the selection operator.

Definition 3.4 (Fitness Function).

A fitness function $F: O \times S \times R \rightarrow [0, 1]$ maps an output o , a specification s and a reference set R to a quality score in $[0, 1]$. The reference set R encodes prior outputs, exemplars or other contextual information that informs the quality judgment. The fitness function is not required to be deterministic: it may be a stochastic discriminator (e.g., a learned model with sampling-based inference).

Definition 3.5 (Discriminative Power).

A fitness function F has discriminative power $\varepsilon > 0$ if for any two outputs $o_1, o_2 \in O$ with true quality gap $|q(o_1) - q(o_2)| > \varepsilon$, the function F ranks them correctly with probability at least $1/2 + \gamma$ for some $\gamma > 0$. Here $q: O \rightarrow [0, 1]$ is a ground-truth quality function.

Theorem 3.4 (Quality Convergence).

If the fitness function F has discriminative power $\varepsilon > 0$ with advantage $\gamma > 0$, and the lattice filter ensures that all survivors are zero-defect, then as the population size $N \rightarrow \infty$, the selected output converges in probability to the quality-optimal zero-defect output.

Specifically, $\Pr[q(\text{Sel}(P, T, F)) > q^* - \varepsilon] \rightarrow 1$ as $N \rightarrow \infty$, where $q^* = \sup\{q(o) : T(o, s) = \text{accept}\}$.

3.5 Computational Complexity

Theorem 3.5 (Verification Cost).

The total verification cost of constrained evolutionary selection is $O(N \cdot C_T)$ per generation, where $C_T = \sum_i C_{\{v_i\}}$ is the cost of evaluating the lattice filter once (the sum of the costs of all individual predicates). For sequential outputs with the rolling-window optimization, C_T is $O(1)$ in the sequence length.

Compare this to exhaustive verification of the output space, which would cost $O(|O| \cdot C_T)$ and is intractable for any nontrivial output space. Constrained evolutionary selection achieves probabilistic completeness (zero-defect selection with probability $1 - \delta$) in $O(\ln(1/\delta) \cdot C_T / \alpha_T)$, which is polynomial in $1/\delta$ and in the lattice evaluation cost.

4 The Centroid Absorber

4.1 The Sequential Coherence Problem

When the generative process produces a sequence of outputs $(o_1, o_2, \dots, o_m)$ rather than a single output, each output must be individually correct *and* coherent with all previous outputs.

Individual correctness is handled by the adjudication lattice. Coherence requires additional machinery.

Definition 4.1 (Coherence).

A sequence $(o_1, \dots, o_m)$ is coherent with respect to a coherence metric $C: O \times O \rightarrow \mathbb{R}_{\geq 0}$ and threshold $\tau > 0$ if for all $i, j \in \{1, \dots, m\}$: $C(o_i, o_j) < \tau$.

The naive approach to coherence validation checks each new candidate against all previous outputs, at cost $O(m)$ per candidate. For long sequences (m in the hundreds or thousands), this pairwise checking dominates the computational budget. We require a mechanism that validates coherence in $O(1)$ time per candidate, independent of sequence length.

4.2 The Absorber Construction

Definition 4.2 (Embedding Function).

Let (E, d_E) be a metric space. An embedding function $\varphi: O \rightarrow E$ maps outputs to points in E such that the metric d_E approximates the coherence metric C : there exist constants $c_1, c_2 > 0$ with $c_1 \cdot C(o, o') \leq d_E(\varphi(o), \varphi(o')) \leq c_2 \cdot C(o, o')$ for all $o, o' \in O$.

Definition 4.3 (Centroid Absorber).

The centroid absorber maintains two statistics over the sequence of accepted outputs: (1) the running centroid $\mu_t = (1/t) \sum_{i=1}^t \varphi(o_i)$, which is the arithmetic mean of the embeddings of all accepted outputs through position t ; and (2) the anchor $\varphi(o_1)$, the embedding of the first accepted output. The absorber validates a new candidate o by checking two conditions: $d_E(\varphi(o), \mu_t) < \tau_c$ (centroid proximity) and $d_E(\varphi(o), \varphi(o_1)) < \tau_a$ (anchor proximity).

The centroid check ensures that the new output is close to the average style of all previous outputs. The anchor check ensures that the new output has not drifted too far from the initial reference point. Both checks execute in $O(1)$ time, requiring only the stored centroid and anchor (not the full history of outputs).

4.3 Drift Bounds

Theorem 4.1 (Centroid Drift Bound).

If each accepted output satisfies $d_E(\varphi(o_t), \mu_{\{t-1\}}) < \tau_c$, then the maximum drift of the centroid from the initial output satisfies $d_E(\mu_t, \mu_1) \leq \tau_c \cdot H_t$, where $H_t = \sum_{i=1}^t 1/i$ is the t -th harmonic number.

Proof.

By the centroid update rule, $\mu_t = ((t-1)/t) \cdot \mu_{\{t-1\}} + (1/t) \cdot \varphi(o_t)$. By the triangle inequality in (E, d_E) , $d_E(\mu_t, \mu_{\{t-1\}}) = (1/t) \cdot d_E(\varphi(o_t), \mu_{\{t-1\}}) < \tau_c/t$. Summing the telescoping bound from $t = 1$ to $t = T$ gives $d_E(\mu_T, \mu_1) \leq \tau_c \cdot \sum_{t=1}^T 1/t = \tau_c \cdot H_T$. For practical sequence lengths ($T < 1000$), $H_T < 7.5$, so the total centroid drift is bounded by approximately $7.5 \cdot \tau_c$.

Y

Theorem 4.2 (Anchor-Centroid Joint Bound).

With both anchor and centroid validation, the maximum deviation of any accepted output from the first output is bounded by $d_E(\varphi(o_t), \varphi(o_1)) \leq \tau_a$ (by construction of the anchor check). Moreover, the population centroid remains within $d_E(\mu_t, \varphi(o_1)) \leq \tau_a$ for all t .

Proof.

The first bound is immediate from the anchor check. For the centroid bound, note that μ_t is a convex combination of $\varphi(o_1), \dots, \varphi(o_t)$. Since each $\varphi(o_i)$ lies within distance τ_a of $\varphi(o_1)$, and the ball of radius τ_a centered at $\varphi(o_1)$ is convex in any normed space, the centroid also lies within this ball. Therefore $d_E(\mu_t, \varphi(o_1)) \leq \tau_a$. This bound is strictly stronger than the centroid-only bound of Theorem 4.1, as it eliminates the harmonic factor entirely.

Y

4.4 The Absorber as a Verification Predicate

Theorem 4.3 (Absorber Soundness).

The centroid absorber with anchor defines a verification predicate $v_C: O \times S \times H \rightarrow \{\text{accept}, \text{reject}\}$, where H is the history (the stored centroid and anchor). This predicate is sound with respect to the coherence defect: $v_C(o, s, H) = \text{accept}$ implies $d_i(o, s) = 0$ for the coherence defect category.

The absorber predicate naturally composes with the adjudication lattice. Since it checks a coherence property that is orthogonal to structural and semantic properties, it adds an independent dimension to the lattice (in the sense of Theorem 3.1), further reducing the joint defect probability.

4.5 Convergence Under the Absorber

Theorem 4.4 (Sequence Convergence).

Constrained evolutionary selection augmented with the centroid absorber converges to a zero-defect, coherent sequence in bounded generations. The required population size N is independent of the sequence length m , because the absorber check executes in $O(1)$ time per candidate.

This result is the key contribution of the absorber construction. Without the absorber, coherence validation requires $O(m)$ comparisons per candidate, making the population size (or the per-candidate cost) grow linearly with sequence length. The absorber reduces this to $O(1)$, ensuring that the convergence guarantees of Theorem 3.3 extend to arbitrarily long sequences with no degradation in population efficiency.

5 The Freshness Predicate

5.1 Temporal Epistemic Consistency

Generative agents operate over time in dynamic environments. The context available to the agent (its epistemic state) may become stale as the environment evolves. An output generated from stale context may be individually correct with respect to its schema and semantics, yet contextually wrong because it reflects an outdated view of the world.

Definition 5.1 (Epistemic State).

The agent's epistemic state at time t is the set $E_t = \{(f, \tau(f)) : f \in F_t\}$, where F_t is the set of facts available to the agent and $\tau(f)$ is the timestamp at which fact f was last observed or updated.

Definition 5.2 (Freshness).

A fact $f \in E_t$ is fresh at time t with respect to threshold θ if and only if $t - \tau(f) < \theta$. The agent's epistemic state is fresh if all facts in E_t that are referenced in the generation of output o are fresh.

5.2 The Freshness Predicate

Definition 5.3 (Freshness Predicate).

The freshness predicate $v_F: O \times E_t \times \theta \rightarrow \{\text{accept}, \text{reject}\}$ accepts output o if and only if every fact in E_t that was referenced during the generation of o is fresh with respect to threshold θ .

The freshness predicate differs fundamentally from other verification predicates. It is not a property of the output alone, nor of the output-specification pair alone. It is a property of the output-context pair: the same output may be fresh at one time and stale at another, depending on when the underlying facts were last updated. This temporal dimension is absent from all structural and semantic predicates.

5.3 Formalization as a Liveness Condition

We frame the freshness requirement as a liveness property in a concurrent system. The agent and the environment are modeled as concurrent processes. The environment process updates facts in a shared mutable store. The agent process reads facts from the store and produces outputs.

In the standard terminology of concurrent systems, freshness corresponds to the liveness condition: "the agent always eventually reads the latest version of every fact it depends on." More precisely, for every fact f that the agent references and every time t at which the environment updates f , there exists a time $t' > t$ with $t' - t < \theta$ at which the agent re-reads f .

Theorem 5.1 (Bounded Temporal Lag).

The freshness predicate v_F with threshold θ ensures that the agent's output reflects the environment state within a bounded time lag of θ . Formally, if $v_F(o, E, \theta) = \text{accept}$ and the output o references facts $f_1, \dots, f_k$, then for each f_i , the version of f_i used in generating o is at most θ time units old.

Theorem 5.2 (Stale Read Prevention).

In a concurrent system with multiple agents $A_1, \dots, A_m$ sharing a mutable fact store, the freshness predicate prevents the stale-read anomaly. Specifically, if agent A_i generates output o at time t and $v_F(o, E_t, \theta) = \text{accept}$, then o does not depend on any fact that was invalidated by another agent more than θ time units before t .

5.4 Composition with the Adjudication Lattice

The freshness predicate is orthogonal to all structural and semantic predicates in the lattice. Structural predicates check *what* the output contains (schema conformance, constraint satisfaction). Semantic predicates check *whether* the output is correct with respect to domain rules. The freshness predicate checks *when* the information used to generate the output was obtained. These are logically independent dimensions.

Theorem 5.3 (Augmented Lattice Guarantees).

An adjudication lattice T augmented with the freshness predicate v_F provides simultaneous correctness (structural and semantic) and timeliness (temporal) guarantees. The joint defect probability of the augmented lattice satisfies $1 - \alpha_{\{T \cup v_F\}} \leq (1 - \alpha_T) + (1 - \alpha_F)$, where α_F is the acceptance probability of the freshness predicate alone. The augmentation incurs no additional asymptotic cost beyond the $O(1)$ freshness check per candidate.

6 The Hierarchical Generation Strategy

6.1 Top-Down Constraint Propagation

Definition 6.1 (Hierarchical Specification Tree).

A hierarchical specification tree is a rooted tree $\Sigma = (V, E)$ where the root $r \in V$ is the top-level specification, interior nodes are sub-specifications and leaves are atomic output specifications. The lattice validates at each level of the hierarchy: an accepted node constrains which children can be generated.

The intuition is that generating a complex structured output is decomposed into a hierarchy of subproblems. The root specification (e.g., "produce a sequence of 100 coherent, schema-conforming outputs") is refined into sub-specifications (e.g., individual output specifications), and each sub-specification is further refined until atomic units are reached. The adjudication lattice validates at each level, and an accepted parent constrains its children.

Theorem 6.1 (Hierarchical Population Reduction).

Hierarchical generation with lattice validation at each level of a d -deep hierarchy reduces the total population size required by a factor exponential in d , compared to flat (non-hierarchical) generation. Specifically, if the acceptance probability at each level is α and the branching factor is b , then flat generation requires $N_{\text{flat}} = O(b^d \cdot \ln(1/\delta) / \alpha)$ candidates, while hierarchical generation requires $N_{\text{hier}} = O(d \cdot b \cdot \ln(1/\delta) / \alpha)$ candidates.

Proof.

In flat generation, the entire composite output must pass the lattice, with acceptance probability approximately α^{b^d} for a d -deep tree with branching factor b . In hierarchical generation, each node is validated independently, with acceptance probability α at each level. The total number of candidates generated is at most $d \cdot b \cdot N_{\text{per_level}}$, where $N_{\text{per_level}} = O(\ln(1/\delta)/\alpha)$. The exponential saving arises because hierarchical decomposition converts a multiplicative probability over b^d components into an additive cost over $d \cdot b$ components. The full proof is given in Appendix A.

Y

6.2 Speculative Generation and Amortized Cost

Definition 6.2 (Speculative Generation).

Speculative generation is the strategy of generating candidate outputs for position $t+1$ before the output at position t has been finalized. If the output at position t is later revised, the speculative candidates for position $t+1$ are discarded (rolled back) and regenerated.

Theorem 6.2 (Amortized Latency).

Speculative generation with rollback reduces the amortized latency to $O(C_G)$ per output, where C_G is the cost of a single generation call. The amortized verification cost is $O(C_T)$ per output. The wasted work from rollbacks is bounded by $p_r \cdot C_G$, where p_r is the probability of upstream revision. Since p_r decreases as the hierarchy stabilizes (by Theorem 3.2), the amortized wasted work decreases over time.

The speculative generation strategy enables pipelining: while the lattice validates the output at position t , the generator produces candidates for position $t+1$. In the common case (no upstream revision), this hides the verification latency entirely behind the generation latency. In the uncommon case (upstream revision), the rollback cost is bounded and decreasing.

6.3 Predictive Resource Allocation via Time-Series Foundation Models

The convergence theorem (Theorem 3.3) and its corollaries establish worst-case bounds on the population size N required for a given failure probability δ . In practice, the framework generates several internal time series as it operates across generations: the joint acceptance probability $\alpha_T(t)$, the per-stratum rejection rates $r_j(t)$, the centroid drift trajectory $d_E(\mu_t, \mu_1)$ and the rollback frequency in speculative generation. These are sequential numerical signals that evolve in characteristic patterns as the system runs. A natural question arises: can these internal signals be forecasted, and can those forecasts be used to allocate computational resources more efficiently than worst-case bounds permit ?

Recent advances in time-series foundation models offer a compelling answer. A pretrained decoder-only forecaster, trained on a large and diverse corpus of temporal data, can produce zero-shot quantile forecasts for previously unseen time series with no task-specific fine-tuning [33]. Such models accept a context window of observed values and output point forecasts together with calibrated quantile estimates (e.g., the 10th through 90th percentiles). We propose that these models can serve as *meta-optimizers* for the adjudication lattice, predicting the trajectory of internal system metrics and informing per-generation resource allocation decisions. Crucially, the forecaster operates outside the verification pipeline: it does not replace or weaken any predicate in the lattice. The deterministic soundness guarantees of the lattice are fully preserved.

Adaptive population sizing. Corollary 3.3.2 establishes that α_T increases monotonically across generations when the generative process conditions on prior accepted outputs. The required population size $N = \lceil \ln(\delta) / \ln(1 - \alpha_T) \rceil$ therefore decreases over time. However, the rate of increase of α_T is not known a priori. By recording $\alpha_T(1)$, $\alpha_T(2)$, ..., $\alpha_T(t)$ and feeding this context to a time-series foundation model, one obtains a forecasted $\alpha_T(t+1)$ with associated quantile bounds. Setting $N(t+1)$ using the conservative (lower) quantile of the forecasted α_T preserves the δ -guarantee while reducing the expected population size relative to the static worst-case bound. The savings are largest in the middle generations, where α_T is increasing rapidly but the worst-case bound has not yet caught up.

Freshness threshold adaptation. The freshness predicate (Section 5) uses a static threshold θ that applies uniformly to all facts in the agent's epistemic state. In practice, fact staleness rates are non-stationary: some facts update on circadian or weekly cycles, others are event-driven with bursty update patterns. A time-series forecaster can model the per-fact update cadence from historical timestamps, producing a forecasted time-to-next-update for each fact f . This enables a variable threshold $\theta(f)$ that reflects the actual dynamics of the environment rather than a single conservative constant. Facts that update rarely can tolerate a larger $\theta(f)$ without loss of timeliness, while facts that update frequently receive a tighter $\theta(f)$. The freshness predicate remains sound under variable thresholds (the proof of Theorem 5.1 carries through with $\theta(f)$ replacing θ), and the computational overhead of the per-fact forecast is amortized over many generation cycles.

Speculative generation scheduling. The amortized cost of speculative generation (Theorem 6.2) depends on the rollback probability p_r . When p_r is high (the hierarchy is still stabilizing), speculation wastes more compute than it saves. When p_r is low (the hierarchy is stable), speculation is highly profitable. A time-series model forecasting p_r from its recent trajectory can trigger speculation only when the predicted p_r falls below a profitability threshold, avoiding wasted generation calls during the volatile early generations. The quantile head of the forecaster provides a natural confidence bound: speculate only when the upper quantile of forecasted p_r is below the threshold, ensuring that the scheduling decision is conservative.

Stratum-level bottleneck detection. The stratum rejection function $r_j(P_t)$ (Definition 3.3) traces a time series for each stratum j across generations. Forecasting these trajectories reveals which strata are converging quickly (rejection rates declining) and which are persistent bottlenecks (rejection rates plateauing). This information can direct the generative process to condition more heavily on outputs that passed the bottleneck strata, or can trigger adaptive tower reconfiguration if a stratum's rejection rate is forecast to remain above a usability threshold.

Formal status of the forecaster. It is essential to clarify what the time-series forecaster is and is not. It is *not* a verification predicate. It is not sound, not deterministic, and not total in the sense of Definition 2.3. It cannot be inserted into the adjudication lattice as a stratum. It is a scheduling oracle: an external advisor that suggests resource allocation parameters (population size, freshness thresholds, speculation decisions) to the framework's control loop. The formal guarantees of the lattice, the absorber and the freshness predicate are unaffected by the accuracy of the forecaster. If the forecaster is wrong, the framework merely uses a suboptimal population size or threshold; it does not admit defective outputs. The worst case is wasted computation, never compromised correctness.

7 Experimental Validation

We validate the theoretical bounds of Sections 3 through 5 using synthetic experiments. All experiments use 10,000 independent trials per configuration unless otherwise noted. Code and raw data are available from the corresponding author upon request.

7.1 Synthetic Adjudication Lattices

We construct synthetic adjudication lattices with known acceptance probabilities to validate the convergence bounds of Theorem 3.3. Each synthetic predicate v_i accepts a candidate independently with probability α_i , corresponding to the idealized case of pairwise orthogonal predicates (Theorem 3.1). We vary the number of predicates $k \in \{2, 4, 6, 8\}$, the per-predicate acceptance probability $\alpha \in \{0.3, 0.5, 0.7, 0.9\}$ and the population size $N \in \{10, 50, 100, 500, 1000\}$. Table 1 reports representative results.

Table 1. Empirical failure rate vs. theoretical bound $(1 - \alpha^k)^N$ over 10,000 trials. Selected configurations shown.

k	α	α_T	N	Theoretical	Empirical
2	0.3	0.0900	10	0.3894	0.3944
2	0.5	0.2500	10	0.0563	0.0591
4	0.3	0.0081	100	0.4434	0.4423
4	0.5	0.0625	100	0.0016	0.0024
6	0.3	0.0007	1000	0.4823	0.4740
6	0.5	0.0156	100	0.2070	0.2142
8	0.3	0.0001	1000	0.9365	0.9338
8	0.5	0.0039	1000	0.0200	0.0216
8	0.9	0.4305	10	0.0036	0.0043

The empirical failure rates match the theoretical bounds to within statistical noise across all 80 configurations tested. For population sizes $N \geq 100$ and per-predicate acceptance probabilities $\alpha \geq 0.5$, the empirical failure rate drops below 10^{-6} (undetectable in 10,000 trials). The results confirm that Theorem 3.3 provides a tight characterization of the required population size.

7.2 Sequential Coherence

We generate sequences of $m = 500$ outputs using a Gaussian generative process in a 128-dimensional embedding space ($d = 128$, $\sigma = 0.5$). The typical inter-point distance in this space is $\sigma\sqrt{2d} \approx 8.0$. We set the centroid threshold $\tau_c = 6.4$ (0.8 of typical distance) and the anchor threshold $\tau_a = 12.0$ (1.5 of typical distance). Table 2 reports results averaged over 200 independent runs.

Table 2. Centroid drift and maximum per-output deviation under three coherence regimes ($m = 500$, $d = 128$, 200 runs). Theoretical bounds from Theorems 4.1 and 4.2.

Condition	Drift (mean)	Drift (std)	Max dev (mean)	Max dev (std)
-----------	--------------	-------------	----------------	---------------

Unconstrained	5.60	0.35	9.27	0.30
Centroid only	3.01	1.84	6.72	2.46
Anchor + centroid	2.39	2.44	7.01	2.25
Bound (centroid)	≤ 43.5			
Bound (anchor)	≤ 12.0		≤ 12.0	

Both constrained conditions reduce centroid drift and maximum deviation relative to the unconstrained baseline. The empirical drift values (3.01 and 2.39) are far below the theoretical worst-case bounds ($\tau_c \cdot H_{500} = 43.5$ for centroid-only and $\tau_a = 12.0$ for anchor-centroid), confirming that the bounds are conservative but valid. The anchor-centroid condition produces the lowest mean drift, consistent with Theorem 4.2.

7.3 Ablation Studies

We ablate individual strata from a synthetic 6-stratum lattice with per-stratum acceptance probability $\alpha = 0.8$ and population size $N = 200$. For each number of removed strata, we measure the defect rate among selected outputs over 10,000 trials. Table 3 reports the results.

Table 3. Defect rate as strata are removed from a 6-stratum lattice ($\alpha = 0.8$, $N = 200$).

Strata removed	Active strata	α_T	Defect rate
0	6	0.2621	0.0000
1	5	0.3277	0.0655
2	4	0.4096	0.1475
3	3	0.5120	0.2496
4	2	0.6400	0.3778
5	1	0.8000	0.5377
6 (no filter)	0	1.0000	0.7385

The defect rate increases monotonically as strata are removed, from zero (full lattice) to 0.74 (no filter). Each removed stratum contributes an independent defect channel, consistent with the multiplicative structure predicted by Theorem 3.1. The full 6-stratum lattice achieves zero observed defects across all 10,000 trials, confirming that even a moderate lattice with $\alpha_T = 0.26$ and $N = 200$ provides practical zero-defect selection.

For the freshness predicate ablation, we introduce artificial staleness at rates ranging from 0% to 80%. Without the freshness predicate, the contextual defect rate equals the staleness fraction (by construction). With the freshness predicate active, all stale outputs are rejected and the defect rate drops to zero at every staleness level, at the cost of an increased rejection rate equal to the staleness fraction. The trade-off is favorable: rejecting stale outputs and regenerating is cheaper than propagating stale information downstream.

7.4 Scaling Analysis

We verify the logarithmic relationship $N = \lceil \ln(\delta) / \ln(1 - \alpha_T) \rceil$ by computing the theoretical N for target failure probabilities $\delta \in \{0.1, 0.01, 0.001, 0.0001\}$ and acceptance probabilities $\alpha_T \in$

$\{0.01, 0.05, 0.1, 0.3, 0.5\}$, then measuring the empirical failure rate at that N over 50,000 trials. Table 4 reports selected results.

Table 4. Population size N required for target failure probability δ , with empirical verification (50,000 trials).

α_T	δ (target)	N (theory)	δ (empirical)	Ratio
0.01	0.01	459	0.0092	0.92
0.05	0.01	90	0.0106	1.06
0.10	0.01	44	0.0097	0.97
0.30	0.01	13	0.0104	1.04
0.50	0.01	7	0.0075	0.75
0.01	0.0001	917	0.0001	1.00
0.10	0.0001	88	0.0001	1.00
0.50	0.0001	14	0.0001	1.00

The empirical failure rates match the theoretical targets consistently: the ratio $\delta_{\text{empirical}}/\delta_{\text{target}}$ is close to 1.0 across all configurations, confirming the tightness of the bound. The population size N grows logarithmically in $1/\delta$ as predicted: for $\alpha_T = 0.10$, moving from $\delta = 0.01$ to $\delta = 0.0001$ (a 100-fold reduction in failure probability) requires only $N = 88$ versus $N = 44$ (a 100% increase in population).

Table 5 reports the wall-clock cost of the centroid absorber check versus pairwise coherence checking as a function of sequence length m , measured in microseconds per candidate on a single CPU core.

Table 5. Absorber check vs. pairwise coherence check timing (microseconds per candidate, $d = 128$).

Sequence length m	Absorber (μs)	Pairwise (μs)	Speedup
10	6.0	10.1	1.7×
100	5.4	30.0	5.5×
1,000	4.7	398.8	85×
5,000	4.7	2,089	445×
10,000	3.9	20,368	5,249×

The absorber check time is constant at approximately 4 to 6 microseconds regardless of sequence length, confirming the $O(1)$ complexity of Theorem 4.4. Pairwise checking grows linearly in m , reaching 20 ms per candidate at $m = 10,000$ (a 5,249× slowdown). For sequences of even moderate length ($m > 100$), the absorber provides orders-of-magnitude speedup.

7.5 Directions for Extended Experiments

The synthetic experiments above validate the mathematical bounds under idealized conditions (independent predicates, Gaussian embeddings, known α_T). Several directions for more elaborate experiments would strengthen the empirical case:

Correlated predicates. Introducing tunable positive correlation between predicate acceptance events (e.g., via a shared latent variable that jointly influences multiple predicates) would empirically validate Observation 3.1 and quantify the degradation from exponential to sub-exponential defect reduction as p increases from 0 to 1.

Learned generator conditioning. Replacing the fixed Gaussian generator with a language model conditioned on prior accepted outputs (via in-context exemplars) would test the approximate conditioning assumption of Theorem 3.2. Measuring whether α_T increases monotonically across generations, or exhibits the non-monotonic behaviour predicted by Observation 3.2, would provide direct evidence on the scope of the self-improvement property.

Real predicate lattices. Constructing a lattice from production-grade predicates (JSON Schema validation, constraint solvers, static analysers, learned semantic classifiers) and running the framework against a real code-generation or structured-data pipeline would measure α_T in a practical setting and reveal whether the population sizes required are computationally feasible.

Long-horizon absorber stress test. Extending the sequential coherence experiment to $m = 100,000+$ outputs would test whether the harmonic drift bound of Theorem 4.1 becomes a practical constraint and whether the anchor-centroid combination (Theorem 4.2) continues to provide tight control at extreme sequence lengths.

Multi-agent freshness. Deploying two or more agents sharing a mutable fact store with variable update rates would test Theorem 5.2 in a concurrent setting and reveal whether the freshness predicate prevents stale-read anomalies at practical clock speeds and network latencies.

8 Related Work

8.1 Formal Verification of Neural Outputs

The verification of neural network properties has attracted sustained attention since the work of Katz et al. [11] on Reluplex and its successors [12, 13]. These approaches verify properties of the network itself (e.g., local robustness around an input point), rather than properties of the output. Abstract interpretation for deep neural networks [14, 15] provides sound overapproximation of network behaviour but does not address output correctness with respect to external specifications. Our framework differs in that it treats the generative model as a black box and verifies the output rather than the model's internal state.

8.2 Evolutionary Computation

Population-based search has a long history in optimization, from genetic algorithms [16, 17] through evolution strategies [18] to modern quality-diversity methods [19, 20]. The covariance matrix adaptation evolution strategy (CMA-ES) [21] is particularly relevant as it adapts the search distribution based on successful candidates, analogous to our conditioned generation (Theorem 3.2). However, classical evolutionary methods optimize a single scalar fitness function and do not incorporate hierarchical deterministic verification. The adjudication lattice adds a qualitative dimension (sound correctness) that complements the quantitative optimization of fitness.

8.3 LLM Output Validation

The recent literature on large language model output validation includes constitutional AI [22], where the model is trained to self-critique; chain-of-thought verification [23], where intermediate reasoning steps are checked; and output classifiers [24, 25], which score outputs along multiple dimensions. These approaches are probabilistic: they reduce the expected defect rate but cannot guarantee zero defects. Our framework subsumes output classifiers as individual predicates in the adjudication lattice and adds the evolutionary population dynamics that provide convergence guarantees.

Several existing techniques share structural similarity with the population-based selection mechanism of the adjudication lattice. Best-of-N sampling generates N candidate outputs from a language model and selects the highest-scoring one according to a reward model. Self-consistency [23] samples multiple chain-of-thought reasoning paths and takes the majority vote. Verifier-augmented generation trains a separate verifier model to score candidate solutions (e.g., in mathematical problem solving) and selects the highest-scoring candidate. Evolutionary program search generates populations of candidate programs and selects those passing test suites. All of these instantiate some variant of "generate many, filter, pick best." The adjudication lattice framework differs from these techniques in three respects. First, it requires the filter to be composed of *deterministic*, *sound* verification predicates rather than learned scoring models,

which provides a qualitatively stronger correctness guarantee (zero residual defect probability rather than reduced expected defect rate). Second, it organizes these predicates into a lattice with explicit orthogonality analysis, yielding quantitative bounds on defect reduction as a function of lattice structure. Third, it adds the centroid absorber and freshness predicate for sequential and temporal consistency, which have no analog in the best-of-N or self-consistency literature. The contribution is thus one of formalization and unification rather than algorithmic novelty: the framework provides provable guarantees for a class of strategies that are currently deployed in ad hoc, guarantee-free fashion.

8.4 Multi-Agent Verification

The formal verification of concurrent and multi-agent systems using temporal logic [26, 27], model checking [28, 29] and process algebra [30] provides the theoretical foundation for our freshness predicate. The liveness condition formulation (Section 5.3) draws directly on the safety-liveness classification of Lamport [31] and the temporal logic framework of Pnueli [32]. Our contribution is to adapt these classical tools to the setting of generative agents, where the "process" under verification is a stochastic generation step rather than a deterministic state transition.

None of the above lines of work combines evolutionary population dynamics with hierarchical deterministic verification and sequential coherence preservation. The adjudication lattice framework unifies all three, providing a single formal framework with provable convergence, coherence and timeliness guarantees.

9 Discussion

9.1 Limitations

The framework rests on several assumptions that, if violated, weaken or invalidate the guarantees. We discuss each in turn, with particular attention to the gap between the formal model and practical deployment.

The soundness gap for semantic predicates. The convergence theorem (Theorem 3.3) assumes that every predicate in the lattice is sound: a predicate that accepts defective outputs undermines the zero-defect guarantee. For structural properties (schema conformance, type checking, constraint satisfaction), sound predicates can be constructed by established methods and verified by construction. For semantic properties (factual correctness, logical consistency, domain-rule compliance in open-ended text), no known technique produces a provably sound and total predicate at acceptable computational cost. This is the central practical limitation of the framework. The lattice provides strong guarantees precisely in the regime where verification is tractable (structured outputs with formal specifications) and weaker guarantees in the regime where verification is hard (open-ended natural-language generation). The framework degrades gracefully under approximate soundness: if a predicate has a false-accept rate of ϵ , the residual defect probability is at most ϵ per stratum (not amplified by the lattice). Practitioners should therefore treat the zero-defect guarantee as an asymptotic ideal that is approached as predicates improve, rather than a property that holds unconditionally in current deployments.

Computational cost for small α_T . The population size bound $N = O(\ln(1/\delta)/\alpha_T)$ is logarithmic in the inverse failure probability but inversely proportional to the joint acceptance probability α_T . For complex specifications with many strata, α_T can be extremely small. If $\alpha_T = 10^{-6}$, achieving $\delta = 0.01$ requires $N \approx 4.6 \times 10^6$ candidates per generation. Each candidate must be generated (cost C_G) and verified (cost C_T), yielding a per-generation cost of $N \cdot (C_G + C_T)$ that may be prohibitive. The hierarchical generation strategy (Section 6) mitigates this by decomposing the specification into a tree and validating at each level independently, converting the multiplicative α_T into an additive cost. Conditioned generation (Theorem 3.2, with the caveats of Observation 3.2) further mitigates the cost by increasing α_T over successive generations. Nevertheless, for specifications where even a single correct output is vanishingly unlikely under the generative distribution, the framework's computational demands can exceed practical budgets.

Positive acceptance probability. Theorem 3.3 requires $\alpha_T > 0$, meaning the generative process must have some positive probability of producing an output that passes the entire lattice. If the generative process is fundamentally incapable of the task (the intersection of the acceptance regions of all predicates is empty under the generative distribution), no population size will help. This is not a limitation of the framework but a reflection of a fundamental impossibility: one cannot verify correctness into existence if the generator cannot produce it.

Discriminative power of the fitness function. The quality convergence result (Theorem 3.4) assumes that the fitness function has positive discriminative power. A random fitness function ($\gamma = 0$) provides no quality improvement; it merely selects a random survivor. In practice, the fitness function may have strong discriminative power for large quality gaps and weak discriminative power for small gaps, leading to diminishing returns as the selected output approaches the quality optimum.

Experimental scope. The experimental validation in Section 7 uses synthetic adjudication lattices with known acceptance probabilities. This validates the mathematical bounds but does not demonstrate the framework's behaviour on real generative agent systems with learned generators and imperfect predicates. Evaluation on production-scale language models, code generators or multi-modal agents is required to assess the practical gap between the formal guarantees and observed system behaviour. We regard such evaluation as essential future work.

9.2 Open Problems

Several directions for future work present themselves.

Adaptive adjudication lattices. Can the lattice learn to tighten predicates based on observed defect patterns? An adaptive lattice that strengthens predicates in defect categories with high observed failure rates could improve convergence without increasing population size. The challenge is to maintain soundness while adapting: a predicate that is tightened too aggressively may reject all candidates, reducing α_T to zero.

Continuous verification predicates. Our framework uses binary predicates (accept/reject). A continuous extension, where each predicate returns a confidence score in $[0, 1]$, would enable softer selection mechanisms (e.g., tournament selection with continuous fitness). The composition theory of continuous predicates (the analog of the lattice structure for binary predicates) has not been developed.

Kolmogorov complexity of the specification. Is there a relationship between the depth of the adjudication lattice required for convergence and the Kolmogorov complexity of the specification? Intuitively, more complex specifications require more verification predicates, but formalizing this intuition would connect the framework to algorithmic information theory.

Infinite-horizon sequential generation. The centroid drift bound of Theorem 4.1 grows as H_t (the harmonic series), which is unbounded though grows slowly. For infinite-horizon generation, the centroid-only bound eventually becomes vacuous. The anchor-centroid bound (Theorem 4.2) avoids this issue, but at the cost of constraining all outputs to a fixed ball around the anchor. Whether there exists a coherence mechanism with sub-logarithmic drift and without a fixed anchor is an open question.

Hardware-accelerated population generation. The computational cost bottleneck identified in Section 9.1 is $N \times (C_G + C_T)$ per generation, where C_G is the cost of generating a candidate and C_T is the cost of verifying it. The mathematical framework is agnostic to the physical

substrate on which generation and verification execute. Advances in photonic computing (optical matrix-vector multiplication via Mach-Zehnder interferometer arrays) could reduce C_G by orders of magnitude by performing the dense linear algebra of generative model inference at the speed of light with near-zero thermal dissipation. Wavelength-division multiplexing enables a single photonic circuit to process hundreds of candidate generations in parallel across distinct wavelengths, potentially collapsing the wall-clock time for population generation from minutes to microseconds even at N in the millions.

The verification cost C_T is less amenable to photonic acceleration because deterministic predicates (schema validators, constraint solvers, temporal logic checkers) rely on branching Boolean logic rather than linear algebra. A hybrid architecture, in which a photonic processing unit handles the generative population (C_G) and passes outputs to parallelized electronic ASICs or FPGAs executing the lattice filter (C_T), is the natural design pattern. Under such a hybrid regime, the speculative generation strategy of Theorem 6.2 becomes dramatically more aggressive: if the marginal energy and time cost of C_G approaches zero, the penalty for speculative rollbacks becomes negligible, allowing the system to speculate dozens of steps into the future across millions of parallel branches with minimal wasted work. Photonic computing cannot change the mathematics of the framework (if $\alpha_T = 0$, no population size helps), but it could collapse the physical time and energy costs of the population draws that the mathematics requires, shifting the framework from economically prohibitive to real-time feasible for complex specifications.

10 Conclusion

We have introduced the adjudication lattice framework for provably correct generative agent outputs. The framework combines three ingredients: a hierarchical lattice of deterministic verification predicates (the adjudication lattice), a centroid absorber for sequential coherence and a freshness predicate for temporal epistemic consistency.

The adjudication lattice provides convergence guarantees: under mild conditions ($\alpha_T > 0$), constrained evolutionary selection converges to a zero-defect output with probability at least $1 - \delta$ for any target failure probability δ , with population size $N = O(\ln(1/\delta))$. The centroid absorber extends these guarantees to coherent sequences, with drift bounds that are independent of sequence length under anchor-centroid validation. The freshness predicate adds temporal consistency, preventing stale-context errors and is orthogonal to all structural and semantic predicates.

Together, these contributions define a unified framework in which the probability of a defective output decreases exponentially with population size and lattice depth, at polynomial computational cost. The framework provides a formal foundation for the design of trustworthy generative agent systems, bridging the gap between probabilistic generation and deterministic verification.

Appendix A Proofs of All Theorems

A.1 Proof of Theorem 3.1 (Exponential Defect Reduction)

Let $v_1, \dots, v_k$ be pairwise orthogonal predicates. Define the events $A_i = \{o : v_i(o, s) = \text{accept}\}$. Orthogonality implies that for each i , the event A_i is not logically determined by any conjunction of the remaining events. We claim that under the stated hypothesis (positive conditional association: for each i and every subset $S \subseteq \{1, \dots, k\} \setminus \{i\}$, $\Pr[A_i \mid \bigcap_{j \in S} A_j] \geq \alpha_i$), the joint probability satisfies:

$$\Pr[\bigcap_i A_i] \geq \prod_i \alpha_i.$$

By the chain rule of conditional probability:

$$\Pr[\bigcap_i A_i] = \prod_i \Pr[A_i \mid \bigcap_{j < i} A_j].$$

The hypothesis states that $\Pr[A_i \mid A_j \text{ for all } j \neq i] \geq \alpha_i$. The inequality $\Pr[A_i \mid \bigcap_{j < i} A_j] \geq \Pr[A_i \mid \bigcap_{j \neq i} A_j]$ does not hold for arbitrary events. We therefore strengthen the hypothesis. Replace the conditional-independence hypothesis in Theorem 3.1 with the following: for each $i \in \{1, \dots, k\}$ and every subset $S \subseteq \{1, \dots, k\} \setminus \{i\}$, the conditional acceptance probability satisfies $\Pr[A_i \mid \bigcap_{j \in S} A_j] \geq \alpha_i > 0$ (positive conditional association). This condition is strictly weaker than mutual independence (under independence, the conditional probability equals α_i exactly) and is satisfied whenever the predicates exhibit positive association in the sense of the FKG inequality. Under this revised hypothesis, the chain rule gives $\Pr[\bigcap_i A_i] = \prod_i \Pr[A_i \mid \bigcap_{j < i} A_j] \geq \prod_i \alpha_i$ directly, with no appeal to dropping conditioning events. The product bound follows.

For the exponential defect reduction, note that the joint defect probability is:

$$1 - \Pr[\bigcap_i A_i] \leq 1 - \prod_i \alpha_i.$$

If each $\alpha_i \geq \alpha$, then $\prod_i \alpha_i \geq \alpha^k$, so $1 - \prod_i \alpha_i \leq 1 - \alpha^k$. For α close to 1 (say $\alpha = 1 - \varepsilon$ for small ε), we have $1 - (1 - \varepsilon)^k \approx k\varepsilon$, which decreases linearly in ε and grows only linearly in k . For α bounded away from 1, the bound $1 - \alpha^k$ decreases exponentially in k . $\square$

A.2 Proof of Theorem 3.2 (Monotone Stratum Rejection)

Let μ_t denote the generative distribution at generation t , and let $\mu_{\{t+1\}}$ denote the distribution obtained by conditioning on the surviving outputs from generation t . Define the feasible set at stratum j as $F_j = \{o : v_i(o, s) = \text{accept for all } i \leq j\}$ and the rejection region at stratum j as $R_j = F_{\{j-1\}} \setminus F_j$.

The stratum rejection function is $r_j(P_t) = |P_t \cap R_j| / |P_t|$. In expectation under μ_t , this equals $\mu_t(R_j)$. Conditioning on surviving outputs shifts mass from rejection regions to acceptance regions. Formally, $\mu_{\{t+1\}}$ is the distribution of $G(s)$ conditioned on outputs that passed stratum j at generation t , so $\mu_{\{t+1\}}(R_j) \leq \mu_t(R_j)$ by the monotonicity of conditional probability under positive conditioning. The data processing inequality for KL divergence provides a quantitative

bound: $D_{\text{KL}}(\mu_{\{t+1\}} \parallel \mu_{T^*}) \leq D_{\text{KL}}(\mu_t \parallel \mu_{T^*})$, where μ_{T^*} is the uniform distribution on the fully feasible set. $\square$

A.3 Proof of Theorem 3.3 (Convergence)

The proof was given in the main text. We provide additional detail on the population size bound. We require $(1 - \alpha_T)^N \leq \delta$. Taking logarithms: $N \cdot \ln(1 - \alpha_T) \leq \ln(\delta)$. Since $\ln(1 - \alpha_T) < 0$ (because $0 < \alpha_T < 1$), dividing by $\ln(1 - \alpha_T)$ reverses the inequality: $N \geq \ln(\delta)/\ln(1 - \alpha_T)$. Using the approximation $\ln(1 - x) \approx -x$ for small x , we obtain $N \geq \ln(1/\delta)/\alpha_T$, confirming that the required population size is logarithmic in $1/\delta$ and inversely proportional to α_T . $\square$

A.4 Proof of Theorem 3.4 (Quality Convergence)

Among the $N \cdot \alpha_T$ survivors (in expectation), the fitness function F ranks them with advantage γ . By a standard Chernoff-type argument on the ranking of the maximum of $N \cdot \alpha_T$ random variables (the quality scores of survivors), the probability that the selected output has quality within ε of the optimum q^* converges to 1 as N grows. Specifically, for each survivor, the probability that F ranks it below a suboptimal output is at most $1/2 - \gamma$. As the number of survivors grows, the probability that all ranking comparisons involving the optimal survivor are incorrect vanishes exponentially. $\square$

A.5 Proof of Theorem 4.1 (Centroid Drift Bound)

We proceed by induction on t . Base case: $t = 1$. The centroid is $\mu_1 = \varphi(o_1)$, so $d_E(\mu_1, \mu_1) = 0 \leq \tau_c \cdot H_1 = \tau_c$. Inductive step: assume $d_E(\mu_{\{t-1\}}, \mu_1) \leq \tau_c \cdot H_{\{t-1\}}$. By the centroid update rule:

$$\mu_t = ((t-1)/t) \cdot \mu_{\{t-1\}} + (1/t) \cdot \varphi(o_t).$$

By the triangle inequality and the convexity of the norm:

$$d_E(\mu_t, \mu_1) \leq ((t-1)/t) \cdot d_E(\mu_{\{t-1\}}, \mu_1) + (1/t) \cdot d_E(\varphi(o_t), \mu_1).$$

Now, $d_E(\varphi(o_t), \mu_1) \leq d_E(\varphi(o_t), \mu_{\{t-1\}}) + d_E(\mu_{\{t-1\}}, \mu_1) < \tau_c + \tau_c \cdot H_{\{t-1\}}$ by the absorber check and the inductive hypothesis. Substituting:

$$\begin{aligned} d_E(\mu_t, \mu_1) &\leq ((t-1)/t) \cdot \tau_c \cdot H_{\{t-1\}} + (1/t) \cdot (\tau_c + \tau_c \cdot H_{\{t-1\}}) = \tau_c \cdot H_{\{t-1\}} + \tau_c/t \\ &= \tau_c \cdot H_t. \quad \square \end{aligned}$$

A.6 Proof of Theorem 4.2 (Anchor-Centroid Joint Bound)

The anchor check requires $d_E(\varphi(o_t), \varphi(o_1)) < \tau_a$ for every accepted output. Thus every embedding lies in the closed ball $B(\varphi(o_1), \tau_a)$. The centroid $\mu_t = (1/t) \sum_{i=1}^t \varphi(o_i)$ is a convex combination of points in $B(\varphi(o_1), \tau_a)$. In any normed vector space, the closed ball is convex, so the centroid also lies in $B(\varphi(o_1), \tau_a)$. Therefore $d_E(\mu_t, \varphi(o_1)) \leq \tau_a$ for all t . $\square$

A.7 Proofs of Theorems 5.1 through 5.3

Theorem 5.1 is immediate from the definition of the freshness predicate. If v_F accepts, every referenced fact f has $t - \tau(f) < \theta$, so the output reflects facts no older than θ time units.

Theorem 5.2 follows from Theorem 5.1 applied to the multi-agent setting. If agent A_i generates output o at time t with $v_F(o, E_t, \theta) = \text{accept}$, then all facts used in generating o are fresh. If agent A_j invalidated fact f at time $t' < t - \theta$, then f is stale and v_F would reject o . The contrapositive gives the desired result.

Theorem 5.3 follows from Theorem 3.1. The freshness predicate is orthogonal to structural and semantic predicates (it checks temporal properties, not content properties). By Theorem 3.1, adding an orthogonal predicate multiplies the acceptance probability, yielding $\alpha_{\{T \cup v_F\}} \geq \alpha_T \cdot \alpha_F$. By the union bound (Boole's inequality), the joint defect probability satisfies $1 - \alpha_T \cdot \alpha_F \leq (1 - \alpha_T) + (1 - \alpha_F)$. $\square$

A.8 Proofs of Theorems 6.1 and 6.2

Theorem 6.1: In flat generation, the composite output consists of b^d atomic components, each requiring acceptance by the lattice. The joint acceptance probability is at most α^{b^d} (assuming independence across components), so the required population size for failure probability δ is $N_{\text{flat}} = \ln(\delta) / \ln(1 - \alpha^{b^d}) \approx \ln(1/\delta) / \alpha^{b^d}$, which is exponential in b^d . In hierarchical generation, each of the d levels has b nodes, each validated independently with population size $N_{\text{level}} = \ln(\delta_{\text{level}}) / \ln(1 - \alpha)$. Setting $\delta_{\text{level}} = \delta / (d \cdot b)$ and applying a union bound yields total failure probability δ . The total number of candidates is $d \cdot b \cdot N_{\text{level}} = O(d \cdot b \cdot \ln(d \cdot b / \delta) / \alpha)$, which is polynomial in all parameters.

Theorem 6.2: Without speculation, each output incurs latency $C_G + C_T$ (generation then verification). With speculation, the generator for position $t+1$ runs concurrently with the verifier for position t . If the output at position t is accepted (probability α_T), the speculative candidates for $t+1$ are valid and the latency is $\max(C_G, C_T)$. If the output at position t is rejected (probability $1 - \alpha_T$), rollback adds an additional C_G . The amortized latency is therefore $\max(C_G, C_T) + (1 - \alpha_T) \cdot C_G$, which is $O(C_G)$ for $C_T \leq C_G$. $\square$

Appendix B The Predicate Lattice for Structured Outputs

We construct a concrete predicate lattice for the domain of structured outputs conforming to a formal schema language (e.g., JSON Schema, XML Schema or an analogous formalism). This construction illustrates the abstract framework of Section 2 with a worked example.

B.1 Schema Validation (Stratum 1)

The base stratum consists of the schema validation predicate v_schema . Given a schema S expressed in a decidable schema language and an output o , $v_schema(o, S) = \text{accept}$ if and only if o conforms to S . For JSON Schema, conformance is decidable in polynomial time in the size of the schema and the output. This predicate is sound by construction: if o conforms to S , then the structural defect $d_struct(o, S) = 0$.

B.2 Constraint Satisfaction (Stratum 2)

The second stratum contains constraint satisfaction predicates. These check domain-specific constraints that go beyond schema conformance. Examples include: uniqueness constraints (no two elements share the same identifier), referential integrity constraints (every reference resolves to an existing element) and range constraints (numeric values lie within specified bounds). Each constraint defines a predicate v_c that is sound with respect to the corresponding defect category. These predicates are generally orthogonal to one another (a uniqueness violation is independent of a range violation) and partially ordered with respect to schema validation (constraint satisfaction implies schema conformance for constraints that reference schema-defined types).

B.3 Forbidden Pattern Detection (Stratum 3)

The third stratum consists of forbidden-pattern predicates. These reject outputs containing known defective patterns, regardless of their schema conformance and constraint satisfaction. Examples include: injection attacks (SQL injection, script injection), encoding errors (invalid UTF-8 sequences, unescaped control characters) and policy violations (presence of prohibited content categories). Forbidden-pattern predicates are orthogonal to constraint satisfaction predicates (an output can satisfy all constraints while containing a forbidden pattern, or violate a constraint while being free of forbidden patterns).

B.4 Cryptographic Signing (Stratum 4)

The fourth stratum consists of the cryptographic signing predicate v_sign , which verifies that the output carries a valid digital signature from a trusted authority. This predicate is the strongest in the lattice: $v_sign(o, s) = \text{accept}$ implies that the output has been reviewed and approved by the signing authority. It is strictly above all other predicates in the partial order. In practice, the signing authority may be an automated verification pipeline that applies all lower-stratum predicates before signing, so that the signature serves as a certificate of compliance.

B.5 The Complete Lattice

The four strata form a graded lattice with the following structure. The bottom element is the trivial predicate $v_{\perp}$ that accepts all outputs. The top element is v_{sign} . Stratum 1 (schema validation) is above $v_{\perp}$. Stratum 2 (constraint satisfaction predicates) contains multiple orthogonal elements, all above stratum 1. Stratum 3 (forbidden patterns) contains multiple orthogonal elements, partially above stratum 2 (some forbidden patterns can only be detected after constraint satisfaction is verified) and partially orthogonal to stratum 2. Stratum 4 (signing) is above all elements in strata 1 through 3. The lattice has width equal to the number of orthogonal predicates in the widest stratum, and height 4.

Appendix C The Category-Theoretic Perspective

This appendix develops the category-theoretic framing sketched in Section 2.6. We construct the relevant categories, functors and natural transformations and show that the adjudication lattice filter arises as a categorical limit.

C.1 The Category **Ver**

Define the category **Ver** as follows. Objects are verification predicates $v: O \times S \rightarrow \{\text{accept}, \text{reject}\}$. A morphism $\alpha: v_1 \rightarrow v_2$ exists if and only if $v_1 \leq v_2$ in the predicate lattice (i.e., v_2 is at least as strict as v_1). Composition is inherited from the transitivity of $\leq$. Identity morphisms exist because $\leq$ is reflexive. The category **Ver** is a thin category (a preorder category): there is at most one morphism between any two objects.

C.2 The Lattice as a Limit

An adjudication lattice $T = (v_1, \dots, v_n)$ defines a finite diagram $D: I \rightarrow \mathbf{Ver}$, where I is the indexing category. For a linearly ordered lattice, I is the finite chain category $[n] = \{1 \rightarrow 2 \rightarrow \dots \rightarrow n\}$. For a lattice with orthogonal predicates, I is a discrete category (no non-identity morphisms).

The lattice filter T is the limit (product) of the diagram D . The limit object in **Ver** is the predicate v_T that accepts output o if and only if every predicate in the diagram accepts o . The limit cone consists of projection morphisms $\pi_i: v_T \rightarrow v_i$, which exist because $v_T \leq v_i$ for all i (the conjunction is at least as strict as each conjunct).

The universal property of the limit states: for any predicate v with morphisms $\alpha_i: v \rightarrow v_i$ for all i (i.e., $v \leq v_i$ for all i), there exists a unique morphism $v \rightarrow v_T$ (i.e., $v \leq v_T$). This is precisely the statement that v_T is the meet (greatest lower bound) of $\{v_1, \dots, v_n\}$ in the predicate lattice.

C.3 The Population Functor

Define the category **Gen** whose objects are natural numbers (generations) and whose morphisms are the unique arrows $n \rightarrow m$ for $n \leq m$. This is the poset category $(\mathbb{N}, \leq)$. Define the functor $\Phi: \mathbf{Gen} \rightarrow \mathbf{FinSet}(O)$ that maps each generation t to the candidate population $P_t \subseteq O$. The morphism $\Phi(t \rightarrow t+1)$ is the selection map that filters P_t through the lattice and selects the survivors.

The selection operator Sel defines a natural transformation from Φ to the constant functor $\Delta_O: \mathbf{Gen} \rightarrow \mathbf{Set}$ that sends every object to O . The naturality condition requires that selection commutes with the generation morphisms: selecting from P_t and then advancing to generation $t+1$ yields the same result as advancing to generation $t+1$ and then selecting. This holds because the lattice filter is deterministic and the fitness function is applied independently at each generation.

C.4 The Absorber as an Endofunctor

The centroid absorber defines an endofunctor $A: \mathbf{Seq}(O) \rightarrow \mathbf{Seq}(O)$ on the category of finite sequences of outputs. The functor A maps a sequence $(o_1, \dots, o_m)$ to the subsequence of coherent outputs, i.e., those that pass the absorber check. The centroid statistics $(\mu_t, \phi(o_1))$ are maintained as part of the functor's internal state. The composition $A \circ \Phi$ gives the full pipeline: generate a population, filter through the lattice and validate coherence.

This functorial perspective, while abstract, clarifies the compositionality of the framework. Each component (lattice, absorber, freshness predicate) is a separate functor or natural transformation and they compose cleanly because their domains and codomains align. The framework's modularity is thus a consequence of its categorical structure.

References

- [1] E. M. Clarke, O. Grumberg and D. A. Peled. Model Checking. MIT Press, 1999.
- [2] L. C. Paulson. Isabelle: A Generic Theorem Prover. Springer, 1994.
- [3] P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In POPL, 1977.
- [4] F. Nielson, H. R. Nielson and C. Hankin. Principles of Program Analysis. Springer, 1999.
- [5] Z. Ji, N. Lee, R. Frieske, et al. Survey of hallucination in natural language generation. ACM Computing Surveys, 55(12), 2023.
- [6] Y. Huang, J. Song, Z. Wang, et al. A survey on hallucination in large language models. arXiv:2311.05232, 2023.
- [7] A. Rebedea, C. Dinu, N. Sreedhar, et al. NeMo Guardrails: A toolkit for controllable and safe LLM applications. In EMNLP (Demo), 2023.
- [8] I. Inan, M. Upasani, J. Chi, et al. Llama Guard: LLM-based input-output safeguard for human-AI conversations. arXiv:2312.06674, 2023.
- [9] P. Christiano, J. Leike, T. Brown, et al. Deep reinforcement learning from human preferences. In NeurIPS, 2017.
- [10] L. Ouyang, J. Wu, X. Jiang, et al. Training language models to follow instructions with human feedback. In NeurIPS, 2022.
- [11] G. Katz, C. Barrett, D. L. Dill, et al. Reluplex: An efficient SMT solver for verifying deep neural networks. In CAV, 2017.
- [12] G. Katz, D. A. Huang, D. Ibeling, et al. The Marabou framework for verification and analysis of deep neural networks. In CAV, 2019.
- [13] C. Brix, M. N. Muller, S. Bak, et al. First three years of the international verification of neural networks competition. International Journal on Software Tools for Technology Transfer, 25, 2023.
- [14] G. Singh, T. Gehr, M. Puschel and M. Vechev. An abstract domain for certifying neural networks. In POPL, 2019.
- [15] M. Mirman, T. Gehr and M. Vechev. Differentiable abstract interpretation for provably robust neural networks. In ICML, 2018.
- [16] J. H. Holland. Adaptation in Natural and Artificial Systems. University of Michigan Press, 1975.
- [17] D. E. Goldberg. Genetic Algorithms in Search, Optimization and Machine Learning. Addison-Wesley, 1989.

- [18] H.-P. Schwefel. *Evolution and Optimum Seeking*. John Wiley & Sons, 1995.
- [19] J.-B. Mouret and J. Clune. Illuminating search spaces by mapping elites. arXiv:1504.04909, 2015.
- [20] A. Cully, J. Clune, D. Tarapore and J.-B. Mouret. Robots that can adapt like animals. *Nature*, 521(7553), 2015.
- [21] N. Hansen and A. Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2), 2001.
- [22] Y. Bai, S. Kadavath, S. Kundu, et al. Constitutional AI: Harmlessness from AI feedback. arXiv:2212.08073, 2022.
- [23] J. Wei, X. Wang, D. Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.
- [24] T. Schick, J. Dwivedi-Yu, R. Dessi, et al. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023.
- [25] N. Jain, S. Vaidyanath, A. Iyer, et al. LLM-assisted code cleaning for training accurate code generators. In *ICLR*, 2024.
- [26] E. A. Emerson. Temporal and modal logic. In *Handbook of Theoretical Computer Science*, Vol. B, 1990.
- [27] M. Y. Vardi. An automata-theoretic approach to linear temporal logic. In *Banff Higher Order Workshop*, 1996.
- [28] G. J. Holzmann. *The SPIN Model Checker: Primer and Reference Manual*. Addison-Wesley, 2004.
- [29] A. Cimatti, E. M. Clarke, E. Giunchiglia, et al. NuSMV 2: An open-source tool for symbolic model checking. In *CAV*, 2002.
- [30] R. Milner. *Communicating and Mobile Systems: The Pi-Calculus*. Cambridge University Press, 1999.
- [31] L. Lamport. Proving the correctness of multiprocess programs. *IEEE Transactions on Software Engineering*, 3(2), 1977.
- [32] A. Pnueli. The temporal logic of programs. In *FOCS*, 1977.
- [33] A. Das, W. Kong, R. Sen and Y. Zhou. A decoder-only foundation model for time-series forecasting. In *ICML*, 2024.

Data Availability Statement

This paper is a theoretical contribution. All results are analytical (proofs of theorems) or based on synthetic experiments with fully specified parameters (Section 7). No external datasets were used. The synthetic experimental configurations (acceptance probabilities, population sizes, embedding dimensions and sequence lengths) are described in sufficient detail in Section 7 to permit independent replication. Code for the synthetic experiments is available from the corresponding author upon request.

Conflicts of Interest

The author declares no conflicts of interest.

Use of AI-Assisted Tools

Portions of this manuscript were drafted with the assistance of large language models: Claude Opus 4.6 (Anthropic), Gemini 3.1 Pro (Google) and Grok 4.2 SuperGrok (xAI). These tools were used for cognitive simplification, prose generation, mathematical exposition and document formatting. All theoretical content, definitions, theorem statements, proof strategies and architectural decisions are the sole intellectual contribution of the author. The author has reviewed the full text for correctness, verified all mathematical claims independently and takes full responsibility for the content of this paper. This disclosure is made in accordance with the COPE position statement on AI-assisted authorship tools.